

Part B: Predictions

	Row-major access	Column-major access
90-deg rotation	4	1
180-deg rotation	2	3

Justification:

1. **90-degree Col-major (Rank 1):** This operation potentially has the best cache performance among the four as column-major access could lead to better spatial locality when rotating the image.
2. **180-degree Row-major (Rank 2):** Here, rows map to rows and columns to columns, so there's a decent level of spatial locality, but not as optimized as column-major due to row-major access.
3. **180-degree Col-major (Rank 3):** Similar to row-major, but column-major access potentially offers better spatial locality, hence a better rank.
4. **90-degree Row-major (Rank 4):** This operation has the worst cache performance since rows in the source map to columns in the destination, leading to strided memory accesses which are cache-unfriendly.

In a 180-degree rotation, elements are accessed in a manner that's more predictable and sequential, which is favorable for cache performance. In a 90-degree rotation, the access pattern is more scattered due to the row-to-column or column-to-row mapping, which can lead to more cache misses. The row-major and column-major distinctions further affect cache performance. Row-major access in a 180-degree rotation is slightly less optimal compared to column-major due to the way data is laid out in memory, leading to different cache hit rates and thus the ranking from 1 to 4.

90-degree Row Major

1. How many addition or subtraction operations are done for each pixel in the image?
 - **2 operations: one addition and one subtraction for computing new indices.**
2. How many multiplication operations are done for each pixel in the image?
 - **0 operations**
3. How many division or modulus operations are done for each pixel in the image?
 - **0 operations**
4. How many comparison operations (equality, less than, and so forth) are done for each pixel in the image, not forgetting any loop-termination conditions? [*]

- **2 comparisons: for loop termination and bounds checking.**
5. How many loads are done for each pixel in the image?
 - **1 load: to fetch the pixel value.**
 6. Of those loads, what fraction hit in the cache?
 - **Ranking 2 (row-major access should offer better spatial locality for loading)**
 7. How many stores are done for each pixel in the image?
 - **1 store: to store the rotated pixel value.**
 8. Of those stores, what fraction are to lines that are already in the cache?
 - **Ranking 4 (storing in column-major order may lead to more cache misses)**

90-degree Column Major

1. How many addition or subtraction operations are done for each pixel in the image?
 - **2 operations: one addition and one subtraction for computing new indices.**
2. How many multiplication operations are done for each pixel in the image?
 - **0 operations**
3. How many division or modulus operations are done for each pixel in the image?
 - **0 operations**
4. How many comparison operations (equality, less than, and so forth) are done for each pixel in the image, not forgetting any loop-termination conditions? [*]
 - **2 comparisons: for loop termination and bounds checking.**
5. How many loads are done for each pixel in the image?
 - **1 load: to fetch the pixel value.**
6. Of those loads, what fraction hit in the cache?
 - **Ranking 4 (column-major access may result in more cache misses for loading)**
7. How many stores are done for each pixel in the image?
 - **1 store: to store the rotated pixel value.**
8. Of those stores, what fraction are to lines that are already in the cache?
 - **Ranking 2 (storing in row-major order should have better spatial locality, hence potentially fewer cache misses)**

180-degree Row Major

1. How many addition or subtraction operations are done for each pixel in the image?
 - **2 operations: one addition and one subtraction for computing new indices.**
2. How many multiplication operations are done for each pixel in the image?
 - **0 operations (no scaling involved)**
3. How many division or modulus operations are done for each pixel in the image?
 - **0 operations**
4. How many comparison operations (equality, less than, and so forth) are done for each pixel in the image, not forgetting any loop-termination conditions? [*]
 - **2 comparisons: for loop termination and bounds checking.**
5. How many loads are done for each pixel in the image?

- **1 load: to fetch the pixel value.**
6. Of those loads, what fraction hit in the cache?
- **Ranking 2 (row-major access benefits spatial locality, but image size exceeds cache size)**
7. How many stores are done for each pixel in the image?
- **1 store: to store the rotated pixel value.**
8. Of those stores, what fraction are to lines that are already in the cache?
- **Ranking 2 (similar reasoning as for loads)**

180-degree Column Major

1. How many addition or subtraction operations are done for each pixel in the image?
- **2 operations: one addition and one subtraction for computing new indices.**
2. How many multiplication operations are done for each pixel in the image?
- **0 operations**
3. How many division or modulus operations are done for each pixel in the image?
- **0 operations**
4. How many comparison operations (equality, less than, and so forth) are done for each pixel in the image, not forgetting any loop-termination conditions? [*]
- **2 comparisons: for loop termination and bounds checking.**
5. How many loads are done for each pixel in the image?
- **1 load: to fetch the pixel value.**
6. Of those loads, what fraction hit in the cache?
- **Ranking 3 (column-major access may hinder spatial locality, leading to more cache misses)**
7. How many stores are done for each pixel in the image?
- **1 store: to store the rotated pixel value.**
8. Of those stores, what fraction are to lines that are already in the cache?
- **Ranking 3 (similar reasoning as for loads)**

	adds/subs	mults	divs/mods	compares	loads	stores
180 row-major	2	1	1	2	2	2
180 col-major	3	1	1	3	3	3
90 row-major	4	1	1	4	4	4
90 col-major	1	1	1	1	1	1